

Aim: write a menu driven program to perform unsorted array operations.

Program:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define capacity 10
```

```
void insertion ( );
```

```
void deletion ( );
```

```
void display ( );
```

```
int A[capacity], size = 0, i;
```

```
void main ( )
```

```
{
```

```
    int ch;
```

```
    char c;
```

```
    do
```

```
{
```

```
    printf ("Unsorted array operations\n");
```

```
    printf ("1. perform insertion\n");
```

```
    printf ("2. perform deletion\n");
```

```
    printf ("3. Display\n");
```

```
    printf ("Enter your choice");
```

```
    scanf ("%d", &ch);
```

```
switch (ch)
{
    case 1: insertion();
    break;
    case 2: deletion();
    break;
    case 3: display();
    break;
    default: printf("Enter correct choice");
}
printf("Do you want to perform one more
operation");
fflush(stdin);
scanf("%c", &c);
}
while (c == 'y' || c == 'Y');
}
void insertion()
{
    int ele;
    if (size < capacity)
    {
        printf("Enter element to insert");
```

```
scanf ("%d", &ele);
```

```
A[size] = ele;
```

```
size++;
```

```
}
```

```
else
```

```
{
```

```
printf ("insertion is not possible");
```

```
}
```

```
}
```

```
void deletion ( )
```

```
{
```

```
int ele, pos;
```

```
printf ("Enter element to delete");
```

```
scanf ("%d", &ele);
```

```
for (i=0; i<=size-1; i++)
```

```
{  
    if (A[i] == ele)
```

```
{  
    pos = i;
```

```
    break;
```

```
}
```

```
}
```

```
for (i=pos+1; i<=size-1; i++)
```

```
{  
    A[i-1] = A[i];
```


}

size--;

}

void display ()

{

for (i=0 ; i<=size-1 ; i++)

printf ("%d", A[i]);

}

~~11~~

Output:

Unsorted array operations

1. perform insertion
2. perform deletion
3. Display

Enter your choice 1

Enter element to insert 26

Do you want to perform one more operation y

Unsorted array operations

1. perform insertion
2. perform deletion
3. Display

Enter your choice 1

Enter element to insert 30

Do you want to perform one more operation y

Enter your choice 3

26 30

Aim: write a menu driven program to perform sorted array operations.

Program:

```
#include <stdio.h>
#include <conio.h>
void insertion ( );
void deletion ( );
void display ( );
#define capacity 10
int A [capacity], size = 0;
void main ( )
{
    int ch; char c;
    do
    {
        printf ("Sorted array operations\n");
        printf ("1. insertion\n");
        printf ("2. deletion\n");
        printf ("3. display\n");
        printf ("Enter your choice (t)");
        scanf ("%d", &ch);
        switch (ch)
        {
```

```
case 1 : insertion ( ) ;  
break ;  
case 2 : deletion ( ) ;  
break ;  
case 3 : display ( ) ;  
break ;  
default : printf ( " Enter correct choice It " );  
}  
printf ( " Do you want to perform one more  
operation It " );  
fflush ( stdin );  
scanf ( " %c " , & c );  
}  
while ( c == 'y' || c == 'Y' );  
}  
void insertion ( )  
{  
    int i, ele, pos ;  
    if ( size < capacity )  
    {  
        printf ( " Enter element to insert " );  
        scanf ( " %d " , & ele );  
        for ( i = 0 ; i <= size - 1 ; i ++ )  
        {
```



```

    if (A[i] > ele)
    {
        pos = i;
        break;
    }
}
for (i = size - 1; i >= pos; i--)
{
    A[i+1] = A[i];
}
A[pos] = ele;
size++;
}
else
{
    printf("insertion not possible");
}
}
}

```

```

void deletion ( )

```

```

{

```

```

    int i, ele, pos;

```

```

    printf("Enter element to delete");

```

```

    scanf("%d", &ele);

```

```

    for (i = 0; i < size - 1; i++)
    {

```

```

        if (A[i] == ele)
        {

```

```


```

```
pos = i;  
break;
```

```
}
```

```
for (i = pos + 1; i <= size - 1; i++)
```

```
{
```

```
    A[i-1] = A[i];
```

```
}
```

```
    size --;
```

```
}
```

```
void display()
```

```
{
```

```
    int i;
```

```
    for (i = 0; i <= size - 1; i++)
```

```
{
```

```
        printf("%d", A[i]);
```

```
}
```

```
}
```


Output:

Sorted Array operations

1. Insertion

2. deletion

3. display

Enter your choice 1

Enter element to insert 26

Do you want to perform one more operation 1

Sorted array operations

1. Insertion

2. deletion

3. display

Enter your choice 3

26

Do you want to perform one more operation 1

Aim: write a menu driven program to perform stack operations.

Program:

```
#include <stdio.h>
#include <conio.h>
void push();
void pop();
void display();
#define capacity 10
int s[capacity], top = -1;
void main()
{
    int ch; char c;
    do
    {
        printf("stack operations\n");
        printf("1. perform push\n");
        printf("2. perform pop\n");
        printf("3. display\n");
        printf("Enter your choice");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1: push();
                    break;
```



```
case 2 : pop();
```

```
break;
```

```
case 3 : display();
```

```
break;
```

```
default: printf("Enter correct choice");
```

```
}
```

```
printf("Do you want to perform another operation
```

```
fflush(stdin);
```

```
scanf("%c", &c);
```

```
}
```

```
while (c == 'y' || c == 'Y');
```

```
{
```

```
void push()
```

```
{
```

```
int ele;
```

```
top = -1;
```

```
{
```

```
if (capacity - 1 == top)
```

```
printf("stack overflow");
```

```
else
```

```
{ printf("Enter element");
```

```
scanf("%d", &ele);
```

```
top = top + 1;
```

```
s[top] = ele;
```

```
}
```

```
}
```

```
void pop ( )
```

```
{ if (top == -1):
```

```
    printf ("stack underflow");
```

```
else:
```

```
{
```

```
    printf ("deleted element is %d", s[top]);
```

```
    top = top - 1;
```

```
}
```

```
}
```

```
void display ( )
```

```
{
```

```
    for (int i = 0; i <= top; i++)
```

```
        printf ("%d", s[i])
```

```
}
```

✓

Output :

Stack operations

1. Perform push
2. perform pop
3. Display

Enter your choice 1

Enter element 50

Do you want to perform another operation y

Stack operations

1. perform push
2. perform pop
3. Display

Enter your choice 3

50

Do you want to perform another operation

Aim: write a program to convert infix to postfix expression.

Program:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <ctype.h>
```

```
int precedence(char);
```

```
void push(char);
```

```
char pop();
```

```
char in[20], post[20], stack[10], x;
```

```
int top = -1, i = 0, j = 0;
```

```
void main()
```

```
{
```

```
printf("Enter infix expression\n");
```

```
gets(in);
```

```
while (in[i] != '\0')
```

```
{ if (isalnum(in[i]))
```

```
{ post[j] = in[i];
```

```
  i++;
```

```
  j++;
```

```
}
```

```
elseif (in[i] == '(')
```

```
{ push(in[i]);
```

```
  i++; }
```

else if (in[i] == ')')

{ while (stack[top] != '(')

{

 x = pop();

 post[j] = x;

 j++;

}

 x = pop();

 i++;

}

else

{

 if (top == -1 || stack[top] == '(')

 push(in[i]);

 else

{

 while (precedence(stack[top]) >= precedence(in[i]))

{

 x = pop();

 post[j] = x;

 j++;

}

 push(in[i]);

}

 i++;

} }

```
while (top != -1)
```

```
{ x = pop();
```

```
post[j] = x;
```

```
j++;
```

```
}
```

```
post[j] = '\0';
```

```
printf("post-fix expression is ");
```

```
puts(post);
```

```
}
```

```
void push(char c)
```

```
{
```

```
top++;
```

```
stack[top] = c;
```

```
}
```

```
char pop()
```

```
{
```

```
char y;
```

```
y = stack[top];
```

```
top--;
```

```
return y;
```

```
}
```

```
int precedence(char z)
```

```
{
```


if (z == '^')

return 3;

else if (z == '*' || z == '/' || z == '%')

return 2;

else if (z == '+' || z == '-')

return 1;

else

return 0;

}

~~14~~
14/11/2024

Output:

Enter infix expression

$2+3-4(5*6/7)-8$

postfix expression is $23+456*7/-8-$

- Enter infix expression

$a+b-c$

postfix expression is $ab+c-$

Aim: write a program to evaluate postfix expression.

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <math.h>

void push(int);
int pop();
int operation(int, int, char);
char post[20];
int s[10], top = -1, i = 0, x, y, z;

void main()
{
    printf("Enter postfix expression\n");
    gets(post);
    while (post[i] != '\0')
    {
        if (isdigit(post[i]))
        {
            push(post[i] - 48);
            i++;
        }
    }
```



```

else
{
y = pop();
x = pop();
z = operation(x, y, post[i]);
push(z);
i++;
}
}

```

```

printf("result = %d", s[top]);
}

```

```

void push(int a)
{
top++;
s[top] = a;
}

```

```

int pop()
{
int b;
b = s[top];
top--;
return b;
}

```

int operation (int x, int y, char c)

{ if (c == '*')

return x * y;

else if (c == '/')

return x / y;

else if (c == '%')

return x % y;

else if (c == '+')

return x + y;

else if (c == '-')

return x - y;

else if (c == '^')

return (pow(x, y));

else

return 0;

}

Output :

Enter postfix expression $53+82-*$

result = 48

Aim: write a program to convert infix to prefix expression.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <conio.h>
```

```
#include <ctype.h>
```

```
int precedence (char);
```

```
void push (char);
```

```
char pop ();
```

```
char in [20], pre [20], stack [10], x;
```

```
int top, = -1, i = 0, j = 0;
```

```
void main ( )
```

```
{
```

```
printf ("Enter infix expression");
```

```
gets (in);
```

```
strrev (in);
```

```
while (in[i] != '\0')
```

```
{ if (isalnum (in[i]))
```

```
{ pre[j] = in[i];
```

```
  i++;
```

```
  j++; }
```

else if (in[i] == ')')

{ push(in[i]);

i++;
}

else if (in[i] == '(')

{ while (stack[top] != ')')

{ x = pop();

Pre[j] = x;

j++;

}

x = pop();

i++;

}

else

{ if (top == -1 || stack[top] == ')')

push(in[i]);

else

{ while (precedence(stack[top]) >= precedence

{ x = pop();

Pre[j] = x;

j++; }

push(in[i]); }

i++;

}

```

while (top != -1)
{
    x = pop();
    pre[j] = x;
    j++;
}
pre[j] = '\0';
printf("prefix expression is");
puts(strrev(pre));
}

void push(char c)
{
    top++;
    stack[top] = c;
}

char pop()
{
    char y;
    y = stack[top];
    top--;
    return y;
}

int precedence(char z)
{
    if (z == '^')
        return 3;
    else if (z == '*' || z == '/' || z == '%')
        return 2;
    else if (z == '+' || z == '-')
        return 1;
    else
        return 0;
}

```


Output :

Enter infix expression

$a+b-c$

Prefix expression is

$+a-bc$

Aim: write a program to evaluate pre-fix expression.

Program:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <math.h>
```

```
void push (int);
```

```
int pop ();
```

```
int operation (int, int, char);
```

```
char pre[20];
```

```
int s[10], top = -1, i = 0, x, y, z;
```

```
void main ( )
```

```
{  
    printf ("Enter prefix expression");
```

```
    gets (pre);
```

```
    strrev (pre);
```

```
    while (pre[i] != '\0')
```

```
    {  
        if (isdigit (pre[i]))
```

```
        {  
            push (pre[i] - 48);
```

```
            i++;
```

```
        }
```

```
        else
```

```
        {
```



```

y = pop();
x = pop();
z = operation(y, x, pre[i]);
push(z);
i++;
}
}
printf("result = %d", s[top]);
}
void push(int a)
{
    top++;
    s[top] = a;
}
int pop()
{
    int b;
    b = s[top];
    top--;
    return b;
}
int operation(int y, int x, char c)
{

```


if (c == '*')

return x * y;

else if (c == '/')

return x / y;

else if (c == '%')

return x % y;

else if (c == '+')

return x + y;

else if (c == '-')

return x - y;

else if (c == '^')

return (pow(y, x));

}

←
return

Output:

Enter prefix expression

2 4 - 3 6 * / 2 +

result = 11

Aim: write a program to perform operations on queue Data structure.

Program:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <conio.h>
```

```
#define capacity 10
```

```
int q[capacity], rear = -1, front = -1;
```

```
void enqueue();
```

```
void dequeue();
```

```
void display();
```

```
void main()
```

```
{ int ch; char c;
```

```
do
```

```
{
```

```
printf("Queue operations");
```

```
printf("1. enqueue\n");
```

```
printf("2. dequeue\n");
```

```
printf("3. display\n");
```

```
printf("Enter your choice");
```

```
scanf("%d", &ch);
```

```
switch (ch)
```

```
{
```



```

case 1: enqueue();
break;
case 2: dequeue();
break;
case 3: display();
break;
default: printf("Enter correct choice");
}
printf("Do you want to perform another operation");
fflush(stdin);
scanf("%c", &c);
} while (c == 'Y' || c == 'y');
}
void enqueue()
{
    int ele;
    if (rear == capacity - 1)
        printf("Queue is full");
    else
    {
        printf("Enter element to insert");
        scanf("%d", &ele);
        rear = rear + 1;
        q[rear] = ele;
        if (front == -1)
            front = 0;
    }
}

```

```
}
```

```
void dequeue ()
```

```
{ if (front == -1 & rear == -1)
```

```
    printf ("Queue is empty");
```

```
else
```

```
{
```

```
    if (front == rear)
```

```
    { printf ("deleted element is %d\n", Q[front]);
```

```
      front = -1; rear = -1;
```

```
    }
```

```
    else
```

```
    {
```

```
        printf ("deleted element is %d\n", Q[front]);
```

```
        front = front + 1;
```

```
    }
```

```
}
```

```
}
```

```
void display ()
```

```
{
```

```
    int i;
```

```
    for (i = front; i <= rear; i++)
```

```
        printf ("%d", Q[i]);
```

```
}
```


Output:

Queue Operations 1. enqueue

2. dequeue

3. display

Enter your choice 1

Enter element to insert 26

Do you want to perform another operation y

Queue operations 1. enqueue

2. dequeue

3. display

Enter your choice 1

Enter element to insert 30

Do you want to perform another operation y

Queue operations 1. enqueue

2. dequeue

3. Display

Enter your choice 2

deleted element is 26

Do you want to perform another operation y

Queue operations 1. enqueue

2. dequeue

3. display

Enter your choice 3

30

Aim: write a program to perform operations on circular queue.

Program:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <conio.h>
```

```
#define capacity 2
```

```
int cq[capacity], rear = -1, front = -1;
```

```
void enqueue();
```

```
void dequeue();
```

```
void display();
```

```
void main()
```

```
{ int ch; char c;
```

```
do
```

```
{ printf("Circular Queue operation\n");
```

```
printf("1. enqueue\n");
```

```
printf("2. dequeue\n");
```

```
printf("3. display\n");
```

```
printf("Enter your choice");
```

```
scanf("%d", &ch);
```

```
switch(ch)
```

```
{
```

```
case 1: enqueue();
```

```

        break;
    case 2: dequeue();
        break;
    case 3: display();
        break;
    default: printf("Enter correct choice");
}
printf("Do you want to perform another opn");
flush(stdin);
scanf("%c", &c);
}
while(c == 'Y' || c == 'y');
}
void enqueue()
{
    int ele;
    if ((rear+1) % capacity == front)
        printf("Circular queue is full");
    else
    {
        printf("Enter element to insert");
        scanf("%d", &ele);
        rear = (rear+1) % capacity;
        cq[rear] = ele;
        if (front == -1)
            front = 0;
    }
}
}

```



```
void dequeue()
```

```
{ if (front == -1 && rear == -1)
    printf("Circular queue is empty");
else
{ if (front == rear)
{ printf("deleted element is %d", cq[front]);
  front = -1;
  rear = -1;
}
else
{ printf("deleted element is %d", cq[front]);
  front = front + 1;
}
}
```

```
void display()
```

```
{ int i;
  for (i = front; i != rear; i = (i + 1) % capacity)
    printf("%d", cq[i]);
  printf("%d", cq[i]);
}
```


Output: Circular Queue operations

1. enqueue

2. dequeue

3. display

Enter your choice 1

Enter element to insert 26

Do you want to perform one more operation y
Circular queue operations

1. enqueue

2. dequeue

3. display

Enter your choice 1

Enter element to insert 30

Do you want to perform one more operation y
Circular queue operations

1. enqueue

2. dequeue

3. display

Enter your choice 3

2630 Do you want to perform one more operation y

Enter your choice 1

Circular queue is full Do you want y

Enter your choice 2

deleted element is 26 Do you want y

Enter your choice 1

Enter element to insert 20 - Do you n

Aim: write a menu-driven program to perform operations on single linked list.

Program:

```
#include <stdio.h>
void createnode();
void insertionatbegin();
void insertionatmiddle();
void insertionatend();
void deletionatbegin();
void deletionatmiddle();
void deletionatend();
void display();
void main()
```

```
{
    int ch;
    char c;
    do
    {
        printf("single linked list operations\n");
        printf("1. create node\n");
        printf("2. insertion at begin\n");
        printf("3. insertion at middle\n");
        printf("4. insertion at end\n");
        printf("5. deletion at begin\n");
```



```
printf("6. deletion at middle\n");  
printf("7. deletion at end\n");  
printf("&display");  
printf("Enter your choice");  
scanf("%d", &ch);  
switch(ch)
```

```
{  
    case 1: createnode();  
    break;  
    case 2: insertionatbegin();  
    break;  
    case 3: insertionatmiddle();  
    break;  
    case 4: insertionatend();  
    break;  
    case 5: deletionatbegin();  
    break;  
    case 6: deletionatmiddle();  
    break;  
    case 7: deletionatend();  
    break;  
    case 8: display();  
    break;  
    default: printf("Enter correct choice");  
}
```


printf("Do you want to perform one more operation

if flush(stdin);

scanf("%c", &c);

{

while (c == 'y' || c == 'Y');

}

struct node

{

int no;

char name[20];

struct node *next;

*head = NULL;

void createnode()

{

struct node *p;

p = (struct node *) malloc(sizeof(struct node));

printf("Enter number");

scanf("%d", &p->no);

p->next = NULL;

head = p;

}

void insertionatbegin()

{

struct node *p;

p = (struct node *) malloc(sizeof(struct node));

```

printf ("Enter number");
scanf ("%d", &p->no);
p->next = head;
head = p;

```

```

}

```

```

void insertion at end()

```

```

{
    struct node *p, *q;

```

```

    p = (struct node *) malloc (size of (struct node));

```

```

    printf ("Enter number");

```

```

    scanf ("%d", &p->no);

```

```

    q = head;

```

```

    while (q->next != NULL)

```

```

    {
        q = q->next;

```

```

    }

```

```

    q->next = p;

```

```

    p->next = NULL;

```

```

}

```

```

void insertion at middle()

```

```

{
    int pos, count = 1;

```

```

    struct node *p, *q;

```

```

    l = (struct node *) malloc (size of (struct node));

```

```

    printf ("Enter no, position");

```

```

    scanf ("%d %d", &p->no, &pos);

```

```

    q = head;

```



```
while (count != pos-1)
```

```
{ q = q->next;
```

```
count++;
```

```
}
```

```
p->next = q->next;
```

```
q->next = p;
```

```
}
```

```
void deletionatbegin()
```

```
{
```

```
struct node *p;
```

```
p = head;
```

```
head = head->next;
```

```
free(p);
```

```
}
```

```
void deletionatend()
```

```
{ struct node *q, *r;
```

```
q = head;
```

```
while (q->next != NULL)
```

```
{
```

```
    r = q;
```

```
    q = q->next;
```

```
}
```

```
r->next = NULL;
```

```
free(q);
```

```
}
```



```
void deletionatmiddle()
```

```
{
```

```
int pos, count = 1;
```

```
struct node *q, *r;
```

```
printf("Enter position");
```

```
scanf("%d", &pos);
```

```
q = head;
```

```
while (count != pos)
```

```
{
```

```
    r = q;
```

```
    q = q->next;
```

```
}
```

```
r->next = q->next;
```

```
free(q);
```

```
}
```

```
void display()
```

```
{
```

```
struct node *q;
```

```
q = head;
```

```
while (q != NULL)
```

```
{
```

```
    printf("%d", q->no);
```

```
    q = q->next;
```

```
}
```

```
}
```

Output:

Single linked list operations

1. Create node
2. Insertion at begin
3. Insertion at middle
4. Insertion at end
5. Deletion at begin
6. Deletion at middle
7. Deletion at end
8. Display

Enter your choice 1

Enter number 10

Do you want to perform one more operation y

Single linked list operations

Enter your choice 2

Enter number 20

Do you - - - y

Single linked list operations

Enter your choice 4

Enter element 30

Do you - - - y

Single linked list operations

Enter your choice 3

Enter no, position 40, 2

Do you want --- y

Single linked list operations

==

Enter your choice 2

20 40 10 30 Do you want --- y

En Single ---

==

Enter your choice 5

Do you want --- y

Single linked list operations

==

Enter your choice 8

40 10 30 Do you want --- n

Aim: write a menu-driven program to perform operations on double linked list.

Program:

```
#include <stdio.h>
#include <malloc.h>
void createnode();
void insertionatbegin();
void insertionatmiddle();
void insertionatend();
void deletionatbegin();
void deletionatmiddle();
void deletionatend();
void display();
void main()
{
    int ch;
    char c;
    do
    {
        printf("\n Double linked list opn\n");
        printf("1. create node\n");
        printf("2. insertion at begin\n");
        printf("3. insertion at middle\n");
        printf("4. insertion at end\n");
```



```

printf("5. deletion at begin\n");
printf("6. deletion at middle\n");
printf("7. deletion at end\n");
printf("8. display");
printf("Enter your choice");
scanf("%d", &ch);
switch(ch)
{
case 1: createnode();
break;
case 2: insertionatbegin();
break;
case 3: insertionatmiddleend();
break;
case 4: insertionatend();
break;
case 5: deletionatbegin();
break;
case 6: deletionatmiddle();
break;
case 7: deletionatend();
break;

```

```
case 8: display();
```

```
break;
```

```
default: printf("Enter correct choice");
```

```
}
```

```
printf("Do u want to perform one more opn");
```

```
fflush(stdin);
```

```
scanf("%c", &c);
```

```
}
```

```
while (c == 'y' || c == 'Y');
```

```
}
```

```
struct node
```

```
{ struct node *prev;
```

```
int no;
```

```
struct node *next;
```

```
*head = NULL, *tail = NULL;
```

```
void createnode()
```

```
{
```

```
struct node *p;
```

```
p = (struct node *) malloc (sizeof (struct node));
```

```
printf("Enter no");
```

```
scanf ("%d", &p->no);
```

```
p->prev = NULL;
```

```
p->next = NULL;
```

```
head = tail = p;
```

```
}
```



```
void insertionatbegin ( )
```

```
{ struct node *p;
```

```
  P = (struct node *) malloc (size of (struct node));
```

```
  printf ("Enter no");
```

```
  scanf ("%d", &p->no);
```

```
  p->prev = NULL;
```

```
  p->next = head;
```

```
  head->prev = p;
```

```
  head = p;
```

```
}
```

```
void insertionatmiddle ( )
```

```
{ struct *p, *q;
```

```
  int pos, count = 1;
```

```
  P = (struct node *) malloc (sizeof (struct node));
```

```
  printf ("Enter no, pos");
```

```
  scanf ("%d %d", &p->no, &pos);
```

```
  q = head;
```

```
  while (count != pos-1)
```

```
  { count++;
```

```
    q = q->next;
```

```
}
```

```
  p->next = q->next;
```

```
  q->next->prev = p;
```

```
  p->prev = q;
```

q → next = p;

}

void insertionatend()

{ struct node *p;

p = (struct node *) malloc(sizeof(struct node));

printf("Enter no");

scanf("%d", &p → no);

p → next = NULL;

p → prev = tail;

tail → next = p;

tail = p;

}

void deletionatbegin()

{ struct node *p;

p = head;

head = head → next;

head → prev = NULL;

free(p);

}

void deletionatmiddle()

{ struct node *p, *q, *r;

int pos, count = 1;

printf("Enter position");

scanf("%d", &pos);


```
q = head;
```

```
while (count != pos)
```

```
{ count++;
```

```
q = q → next;
```

```
}
```

```
p = p → prev;
```

```
r = q → next;
```

```
p → next = r;
```

```
q → next = p;
```

```
free(q);
```

```
}
```

```
void deletionatend()
```

```
{
```

```
struct node *p;
```

```
p = tail;
```

```
tail = tail → prev;
```

```
tail → next = NULL;
```

```
free(p);
```

```
}
```

```
void display()
```

```
{
```

```
struct node *p;
```

```
p = head;
```

```
while (p != NULL)
```

```
{ printf("%d", p → no);
```

```
p = p → next;
```

```
}
```

```
}
```


Output: Double linked list opn

≡

Enter your choice 1

Enter number 11

Do you want to perform another opn y

Double linked list opn

≡

Enter your choice 2

Enter number 22

Do you - - - - y

Double linked list opn

≡

Enter your choice 3

Enter no 33

Do you - - - - y

Double linked list opn

≡

Enter your choice 8

22 11 33 Do you - - - - y

Double linked list opn

≡

Enter your choice 7

Enter pos 2

Do you - - - - y

Double linked list opn

Enter your choice 8

22 33

Do you want to perform another opn

n

Aim: Write a menu driven program to perform operations on circular single linked list.

Program:

```
#include <stdio.h>
#include <conio.h>
#include <malloc.h>
#include <ctype.h>

void createnode();
void insertionatbegin();
void insertionatend();
void insertionatmiddle();
void deleteinsertionatbegin();
void deletionatend();
void deletionatmiddle();
void display();

struct node
{
    int no;
    struct node *next;
} *head = NULL;

void main()
{
    int ch;
    char c;
    do
    {
        printf("Circular single linked list operations\n");
        printf("1. create node\n");
```



```

printf ("2. insertion at begin\n");
printf ("3. insertion at end\n");
printf ("4. insertion at middle\n");
printf ("5. deletion at begin\n");
printf ("6. deletion at end\n");
printf ("7. deletion at middle\n");
printf ("8. display\n");
printf ("Enter your choice");
scanf ("%d", &ch);
switch (ch)
{
    case 1: createnode();
        break;
    case 2: insertionatbegin();
        break;
    case 3: insertionatend();
        break;
    case 4: dinsertionatmiddle();
        break;
    case 5: deletionatbegin();
        break;
    case 6: deletionatend();
        break;
    case 7: deletionatmiddle();
        break;
    case 8: display();
        break;
}

```



```

        default: printf("Enter correct choice");
    }
    printf("Do u want to perform another operation");
    fflush(stdin);
    scanf("%c", &c);
}
while (c == 'y' || c == 'Y');
}
void createnode()
{
    struct node *p;
    p = (struct node *) malloc(sizeof(struct node));
    printf("Enter no");
    scanf("%d", &p->no);
    p->next = p;
    head = p;
}
void insertionatbegin()
{
    struct node *p, *q;
    p = (struct node *) malloc(sizeof(struct node));
    printf("Enter no");
    scanf("%d", &p->no);
    p->next = head;
    q = head;
    while (q->next != head)
    {
        q = q->next;
    }
}

```


q → next = p;

head = p;

}

void insertionatend()

{ struct node *p, *q;

p = (struct node *) malloc (size of (struct node));

printf ("Enter no");

scanf ("%d", &p → no);

q = head;

while (q → next != head)

{ q = q → next;

}

q → next = p;

p → next = head;

}

void insertionatmiddle()

{ struct node *p, *q;

int pos, count = 1;

p = (struct node *) malloc (size of (struct node));

printf ("Enter no, pos");

scanf ("%d %d", &p → no, &pos);

q = head;

while (count != pos - 1)

{

count ++;

```
q = q->next;
```

```
}
```

```
p->next = q->next;
```

```
q->next = p;
```

```
}
```

```
void deletionatbegin()
```

```
{
```

```
struct node *p, *q;
```

```
p = head;
```

```
q = head;
```

```
while (q->next != head)
```

```
{  
    q = q->next;
```

```
}
```

```
q->next = head->next;
```

```
head = head->next;
```

```
free(p);
```

```
}
```

```
void deletionatend()
```

```
{ struct node *q, *r;
```

```
q = head;
```

```
while (q->next != head)
```

```
{  
    r = q;
```

```
    q = q->next;
```

```
}
```

```
r->next = head;
```

```
free(q);
```

```
}
```



```
void deletionatmiddle()
```

```
{ int pos, count = 1;
```

```
struct node *q, *r;
```

```
printf("Enter position");
```

```
scanf("%d", &pos);
```

```
q = head;
```

```
while(count != pos)
```

```
{ r = q;
```

```
q = q->next;
```

```
count++;
```

```
}
```

```
r->next = q->next;
```

```
free(q);
```

```
}
```

```
void display()
```

```
{
```

```
struct node *q;
```

```
q = head;
```

```
while(q->next != head)
```

```
{
```

```
printf("%d", q->no);
```

```
q = q->next;
```

```
}
```

```
printf("%d", q->no);
```

```
}
```

O/p: Circular single linked list operations

Enter your choice 1

Enter no 30

Do you - - - y

Circular single - - -

Enter your choice 2

Enter no 20

Do you - - - y

Circular single - - -

Enter your choice 3

Enter no 40

Do you - - - y

Enter your choice 4

Enter no, pos 10 2

Do you - - - y

Circular single - - -

Enter your choice 8

20 10 30 40

Do you want to perform one more opn n

Aim: write a menu driven program to perform operations on circular double linked list.

Program:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <malloc.h>
```

```
void createnode();
```

```
void insertionatbegin();
```

```
void insertionatend();
```

```
void insertionatmiddle();
```

```
void deletionatbegin();
```

```
void deletionatend();
```

```
void deletionatmiddle();
```

```
void display();
```

```
struct node
```

```
{  
    struct node *prev;
```

```
    int no;
```

```
    struct node *next;
```

```
} *head = NULL, *tail = NULL;
```

```
void main()
```

```
{  
    int ch;
```

```
    char c;
```

```
    do
```

```
    {
```

```

printf("Circular double linked list operations\n");
printf("1. createnode\n");
printf("2. insertionatbegin\n");
printf("3. insertionatend\n");
printf("4. insertionatmiddle\n");
printf("5. deletionatbegin\n");
printf("6. deletionatend\n");
printf("7. deletionatmiddle\n");
printf("8. display\n");
printf("Enter your choice");
scanf("%d", &ch);
switch(ch)
{
    case 1: createnode();
    break;
    case 2: insertionatbegin();
    break;
    case 3: insertionatend();
    break;
    case 4: insertionatmiddle();
    break;
    case 5: deletionatbegin();
    break;
    case 6: deletionatend();
    break;
}

```


case 7: deletionatmiddle()

break;

case 8: display()

break;

default: printf("Enter correct choice");

}

printf("Do u want to perform another op");

flush(stdin);

scanf("%c", &c);

}

while (c == 'y' || c == 'Y');

}

void createnode()

{ struct node *p;

p = (struct node *) malloc(sizeof(struct node));

printf("Enter no");

scanf("%d", &p->no);

p->prev = p;

p->next = p;

head = tail = p;

}

void insertionatbegin()

{

struct node *p;

p = (struct node *) malloc(sizeof(struct node));

printf("Enter number");

```
scanf("%d", &p->no);
```

```
p->next = head;
```

```
head->prev = p;
```

```
p->prev = tail;
```

```
tail->next = p;
```

```
head = p;
```

```
}
```

```
void insertionatend()
```

```
{
```

```
    struct node *p;
```

```
    p = (struct node *) malloc(sizeof(struct node));
```

```
    printf("Enter no");
```

```
    scanf("%d", &p->no);
```

```
    tail->next = p;
```

```
    p->prev = tail;
```

```
    p->next = head;
```

```
    head->prev = p;
```

```
    tail = p;
```

```
}
```

```
void insertionatmiddle()
```

```
{    int pos, count = 1;
```

```
    struct node *p, *q;
```

```
    p = (struct node *) malloc(sizeof(struct node));
```

```
    printf("Enter no, pos");
```

```
    scanf("%d %d", &p->no, &pos);
```

```
    while (count != pos-1)
```



```
{ q = q → next ;
```

```
count ++;
```

```
}
```

```
P → next = q → next
```

```
q → next → prev = P;
```

```
P → prev = q;
```

```
q → next = P;
```

```
}
```

```
void deletionatbegin()
```

```
{ struct node *p;
```

```
P = head;
```

```
head = head → next;
```

```
tail → next = head;
```

```
head → prev = tail;
```

```
free(p);
```

```
}
```

```
void deletionatend()
```

```
{
```

```
struct node *p;
```

```
p = tail;
```

```
tail = tail → prev;
```

```
tail → next = head;
```

```
head → prev = tail;
```

```
free(p);
```

```
}
```

```
void deletionatmiddle()
```

```
{  
    int pos, count = 1;  
    struct node *p, *q, *r;  
    printf("Enter position");  
    scanf("%d", &pos);  
    q = head;  
    while (count != pos)  
    {  
        q = q->next;  
        count++;  
    }  
    p = q->prev;  
    r = q->next;  
    p->next = r;  
    r->prev = p;  
    free(q);  
}
```

```
void display()
```

```
{  
    struct node *p;  
    p = head;  
    while (p->next != head)  
    {  
        printf("%d", p->no);  
        p = p->next;  
    }  
    printf("%d", p->no);  
}
```


Output: circular double linked list operations

1. Create node

==

Enter your choice 1

Enter number 61

Do you - - - - y

Circular double - - - -

==

Enter your choice 2

Enter number 59

Do you want - - - - y

Circular double - - - -

==

Enter your choice 4

Enter no, pos 60 2

Do you want to - - - - y

Circular double - - - -

==

Enter your choice 8

59 60 61

Do you - - - - n

Aim: write a c program to implement linear search.

Program:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
void main()
{
    int a[5] = {1, 2, 3, 4, 5};
    int key = 5, r;
    r = linearsearch(a, key);
    if (r == -1)
    {
        printf("search is unsuccessful");
    }
    else
    {
        printf("search is successful at index %d", r);
    }
}

int linearsearch(int a[], int key)
{
    int i;
    for (i = 0; i < 5; i++)
    {
        if (a[i] == key)
        {
            return i;
        }
    }
    return -1;
}
```

Output: Search is successful at index 4

Aim: write a "C" program to implement binary search.

Program:

```
#include <stdio.h>
#include <math.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int key = 50;
    char r;
    int n = 5;
    u = binarysearch(a, key, 0, n-1);
    if (u == -1)
    {
        printf("search is unsuccessful");
    }
    else
    {
        printf("search is successful at index %d", r);
    }
    return 0;
}

int binarysearch(int a[], int key, int low, int high)
{
    int mid;
    while (low <= high)
    {
        mid = floor((low + high) / 2);
```

```
if (key == a[mid])  
    return mid;  
else if (key > a[mid])  
    low = mid + 1;  
else  
    high = mid - 1;  
}  
return -1;  
}
```

Output:

Search is successful at index 4

Aim: write a C program to implement bubble sort.

Program:

```
#include <stdio.h>

void bubblesort(int[], int);

int main()
{
    int i, a[5] = {14, 15, 12, 16, 13};
    clrscr();
    bubblesort(a, 5);
    printf("Elements after sorting\n");
    for (i = 0; i < 5; i++)
    {
        printf("%d\n", a[i]);
    }
    return 0;
}

void bubblesort(int a[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

Output:

Elements after sorting

12

13

14

15

16

Aim: write a C program to implement selection sort.

Program:

```
#include <stdio.h>
void selectionsort (int [], int);
int main()
{
    int i, a[5] = {10, 16, 9, 14, 12};
    clrscr();
    selectionsort(a, 5);
    printf("Elements after sorting\n");
    for (i=0; i<5; i++)
    {
        printf("%d\n", a[i]);
    }
    return 0;
}

void selectionsort (int a[], int n)
{
    int i, temp, j, min;
    for (i=0; i<=n-1; i++)
    {
        int min=i;
        for (j=i+1; j<=n-1; j++)
        {
            if (a[min]>a[j])
            {
                min=j;
            }
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}
```

Output:

Elements after sorting

9
10
12
14
16